TheTowerDNN and GraphRanker were both trained using contrastive learning and HPO objective metrics that were partitioned into head, torso and tail movies (tier=0,1,2, resp) as defined by the training positive dataset movie ratings frequency.

- The movie embeddings generated from the TwoTowerDNN Candidate model show decent separation of clusters in the t-SNE and UMAP plots with spatially overlapping tier=0,1,2. They look healthy
- The train, val learning curves for ndcg_<tier> in the TwoTowerDNN and GraphRanker training all show healthy learning above random change metrics.
- The batch sizes have been adjusted to >= 1024 in order to be able to learn the movie tail distribution. Binomial distribution and Poisson distribution were to calculate the batch sizes. See recommender_systems/docs/mlops/movie_tier_counts.txt (add github ref).
- End-to-end inference metrics using the trained models and the test dataset and movie tiers defined by frequency of movie ratings in train positives:
    - 48_285 test dataset ratings
    - 3_883 num_catalog_movies
    - "ndcg_head_20": 0.044804782414759374,
    - "ndcg_torso_20": 0.02441173223123198,
    - "ndcg_tail_20": 0.0060162138110536065,
        - "random_ndcg_head_20": 0.003064348921321239,
        - "random_ndcg_torso_20": 0.0028287642332191837,
        - "random_ndcg_tail_20": 0.0020430092333822906,
    - "recall_head_20": 0.0566528978866582,
    - "recall_torso_20": 0.052628152590776876
    - "recall_tail_20": 0.018960269097068005,
        - "random_recall_head_20": 0.0051841404718014975,
        - "random_recall_torso_20": 0.005218351778443615,
        - "random_recall_tail_20": 0.005150656708730404,
    - "catalog coverage": 0.7334535121917725,
    - "count_users_head_20": 4584,
    - "count_users_torso_20": 4063,
    - "count_users_tail_20": 731,
    - "count_movie_catalog_head": 565,
    - "count_movie_catalog_torso": 1709,
    - "count_movie_catalog_tail": 1609,
    - "count_test_head": 565,
    - "count_test_torso": 1692,
    - "count_test_tail": 470,
    - "frac_pred_tiers_head_20": 0.37626527050610753,
    - "frac_pred_tiers_torso_20": 0.5706128476495198,
    - "frac_pred_tiers_tail_20": 0.1140902872777015
- =>
  count_movie=np.array([565, 1709, 1609])
  count_test=np.array([565, 1692, 470])
  count_users=np.array([4584, 4063, 731])
  count_movie/count_movie.sum() = array([0.1455, 0.4401, 0.4144])
  count_users/count_users.sum() =array([0.4888, 0.4332, 0.0780])
  count_test/count_test.sum() = array([0.2072, 0.6205, 0.1724])
- =>

    - "frac_pred_tiers_tail_20": 0.11409
        - 0.114* k=20 yields 2.28 niche (tail) movies" for users who rated tier=2 movies in test ("niche movie users")
        - This is above 0 so is predicting niche items to niche users, but is it different than random selection? Yes, uniform random selection is count_movie/count_movie.sum() = array([0.1455, 0.4401, 0.4144]). So 20 uniform random recommendations * [0.1455, 0.4401, 0.4144] = *[2.9, 8.8, 8.3]*.
          The frac_pred_tiers = 20 * [0.376, 0.571, 0.114] = *[7.5, 11.4, 2.3]* which is clearly not the uniform random distribution. So this is more evidence that the end-to-end inference is able to make niche recommendations.
- =>
    - The ndcg_<tier> and recal_<tier> metrics are all 3 to 10 times the metric expected from random selection, which shows that the models successfully predict niche movies for niche users while still predicting popular movies to popular users.
- ⇒
    - "catalog coverage": 0.73
      The recommendation pipeline is recommending to users, far more thant the small number of very popular movies, hence is overcoming the popular movie bias.

Could the niche user/movie recommendations be improved by increasing num_candidates to be 10x to 100x top_k?

- I took the intersection of users who rated tier=2 movies in the train, val, and positive datasets, which is 222 users. For those users, ANN searches w.r.t. each dataset (using their first timestamp in the dataset) were made to get the top k number of recommendations. The average number of movie tiers in each set of top k recommendations was determined. Here are those averages:
  dataset_k—------avg counts in the k—---------- avg counts / k
  test_00070: 21.432 : 0.306
  test_00100: 31.914 : 0.319
  test_00200: 69.311 : 0.347
  test_00300: 108.378 : 0.361
  test_00400: 149.604 : 0.374
  test_00500: 190.964 : 0.382
  test_00600: 232.829 : 0.388
  test_00700: 275.928 : 0.394
  test_00800: 318.568 : 0.398
  test_00900: 362.009 : 0.402
  test_01000: 405.973 : 0.406
  test_01100: 449.500 : 0.409
  test_01200: 492.847 : 0.411
  test_01300: 535.955 : 0.412
  test_01400: 579.090 : 0.414
  test_01500: 622.946 : 0.415
  test_01600: 666.887 : 0.417
  test_01700: 711.126 : 0.418
  test_01800: 755.176 : 0.420
  test_01900: 800.099 : 0.421

test_02000: 844.378 : 0.422
train_00070: 20.802 : 0.297
train_00100: 31.104 : 0.311
train_00200: 67.811 : 0.339
train_00300: 105.662 : 0.352
train_00400: 145.113 : 0.363
train_00500: 185.563 : 0.371
train_00600: 226.644 : 0.378
train_00700: 268.437 : 0.383
train_00800: 310.568 : 0.388
train_00900: 353.482 : 0.393
train_01000: 396.707 : 0.397
train_01100: 439.568 : 0.400
train_01200: 483.378 : 0.403
train_01300: 526.806 : 0.405
train_01400: 570.216 : 0.407
train_01500: 614.586 : 0.410
train_01600: 658.216 : 0.411
train_01700: 702.824 : 0.413
train_01800: 747.311 : 0.415
train_01900: 792.032 : 0.417
train_02000: 836.432 : 0.418
val_00070: 21.095 : 0.301
val_00100: 31.500 : 0.315
val_00200: 68.374 : 0.342
val_00300: 106.865 : 0.356
val_00400: 147.279 : 0.368
val_00500: 188.306 : 0.377
val_00600: 229.671 : 0.383
val_00700: 272.131 : 0.389
val_00800: 314.671 : 0.393
val_00900: 357.806 : 0.398
val_01000: 401.464 : 0.401
val_01100: 445.036 : 0.405
val_01200: 488.315 : 0.407
val_01300: 531.275 : 0.409
val_01400: 574.622 : 0.410
val_01500: 618.757 : 0.413
val_01600: 662.662 : 0.414
val_01700: 706.874 : 0.416
val_01800: 751.513 : 0.418
val_01900: 796.013 : 0.419
val_02000: 840.189 : 0.420

- ⇒ this shows that increasing the number of candidates would increase the fraction of tier=2 movies in the recommendations.
    - Increasing num_candidates increases the graph size by by factor if candidate increase. E.g. the metrics at top of file are from num_candidates=70 model. If we increase the number of candidates to 2000 the factor of increase is nearly 30.

        - In the GraphRanker, the graph of senders and receivers given to the GATV2 uses a star bipartite morphology centered around the user node and so the memory scaling is a linear O(c) so the VRAM needed will scale linearly too.
        - The VRAM consumed for the num_candidates=70 model was roughly 900 MiB. so scaling up to 200 candidates would used about 2.6GiB of the available 30 GB RAM.
        - The runtime complexity is O(E+V) and over a batch size of 1024 a model training using num_candidates=200 would be roughly 2-3 times longer than a model trained with num_candidates=70.
        - num_candidates=70 was enough to implement niche user recommendations.
            - Increasing num_candidates to 2000 would give a moderate increase in the fraction of tier=2 recommendations returned to niche users, but at the cost of nearly 20 times the elapsed runtime.
              (1024*(50+2000)) / (1024*(50+70))

- End-to-end inference metrics using the trained models and the test dataset and user tiers defined by lengths of movie ratings from full train and val datasets:
    - 48_285 test dataset ratings
    - 6_040 num_catalog_users
    - "count_users_head_20": 1045,
    - "count_users_torso_20": 3087,
    - "count_users_tail_20": 964,
    - "ndcg_head_20": 0.06260424300544941,
    - "ndcg_torso_20": 0.041394163820992806,
    - "ndcg_tail_20": 0.04262620676019379,
    - "random_ndcg_head_20": 0.006235896173105689,
    - "random_ndcg_torso_20": 0.0034036375933420866,
    - "random_ndcg_tail_20": 0.0023569452032742065,
    - "random_recall_head_20": 0.006737502572247862,
    - "random_recall_torso_20": 0.005158498654480724,
    - "random_recall_tail_20": 0.005150656708730426,
    - "recall_head_20": 0.0542801545501636,
    - "recall_torso_20": 0.053974246501558194
    - "recall_tail_20": 0.08082641770401108,
    - =>
        - The ndcg_<tier> and recall_<tier> metrics for even the sparse (tail) users are better than random by a factor of about 20 showing that the model's GATV2 multi-hop has learned enough interaction to recommend even when there are not many ratings in a user's history.